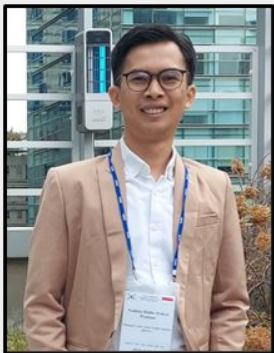


Unlocking Secrets: Can Python **Reveal the Strengths** of LLMs in Cryptanalysis?

Nadhia **Ridho Wahyu** Pradana



Pycon Busan
2026



Nadhiar Ridho Wahyu Pradana

Master's of ICT Policy Student at Handong Global University

Pohang, KR / South Jakarta, ID

nadhlar.live / nadhlar@handong.ac.kr

“

"If you are not networking, you are not working"



Pycon Busan
2026



Can You Read This Message ??

Khoor, Zruog! Fdq BRX Uhdg Wklv?

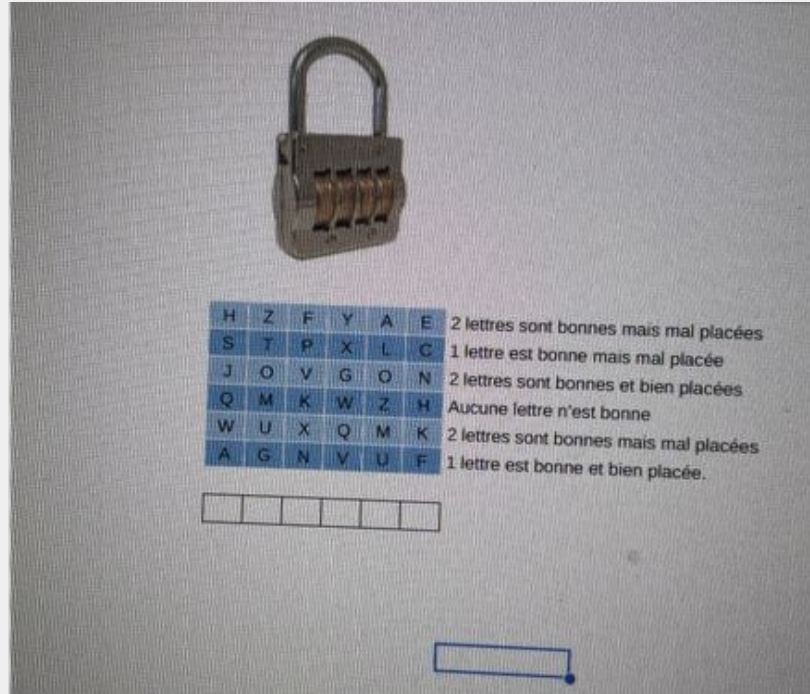
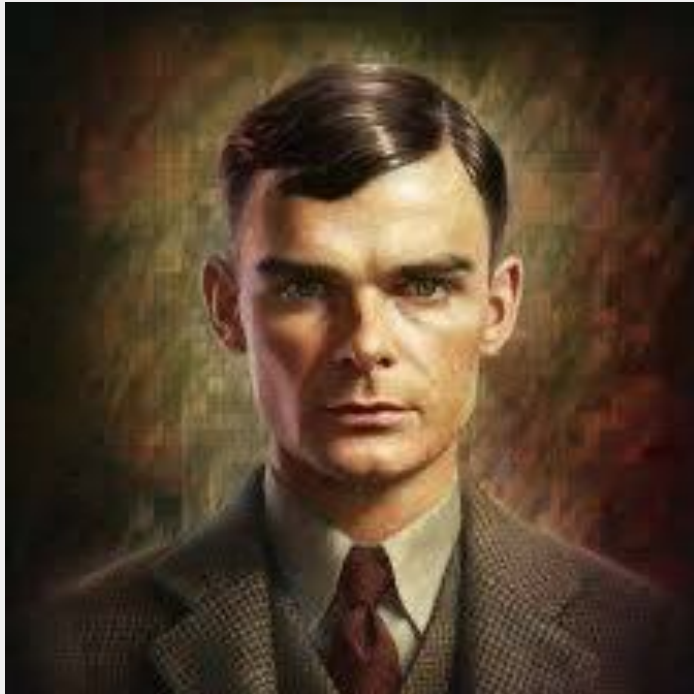
Hello, World! Can YOU Read This?

Caesar Cipher, Shift 3





What do you know about Cryptanalysis ??





Background



Cryptography is Everywhere

Every HTTPS request, every token, every transaction protected by ciphers.



LLMs are Reasoning Engines

GPT-4o, Claude, Gemini now solve math, write code, and prove theorems.

The Big Question:

Can LLMs reason through cryptographic patterns without being explicitly taught?

13

Cipher algorithms
tested

5+

LLM models
compared





Research Objectives

RQ1

Benchmark cryptanalysis ability of frontier LLMs across difficulty tiers

RQ2

Quantify the impact of prompting strategy on decryption accuracy

RQ3

Compare proprietary vs. open-source model performance on cipher tasks

RQ4

Identify failure modes: when do LLMs recognize ciphers but still fail?



Pycon Busan
2026

All experiments orchestrated, measured, and visualized using Python



Cipher Taxonomy

EASY

- Caesar (shift-3)
- ROT13
- Atbash
- Reverse
- Morse Code

*No key needed · Pattern recognition
· Pre-training exposure likely*

MEDIUM

- Vigenère (keyword)
- Rail Fence (3 rails)
- Swap Pairs
- Polybius Square

*Key or parameter required ·
Multi-step reasoning · Moderate
complexity*

HARD

- Bacon (5-bit A/B)
- WordShift
- XOR (hex output)
- RSA small-n

*Mathematical reasoning ·
Multi-step algorithm · Novel
encoding patterns*

7



Pycon Busan
2026

Advanced : AES, DES, etc

*This presentation does **not** cover advanced cipher techniques.



How Python powers every layer of PyCryptoLLM

“



PyCryptoLLM: 5-Layer Framework

Layer 1 · Cipher Bank

ciphers.py · CipherFactory · 13 algorithms · Easy / Medium / Hard

Layer 2 · Dataset Builder

dataset.py · 30 texts × 9 domains · auto-encrypt → CipherSample

Layer 3 · Prompt Engine

prompt_engine.py · 4 modes · `<r>text</r>` extraction · near_exact_match

Layer 4 · LLM Registry

llm_clients.py · Closed-sources LLM and Open Source LLM

Layer 5 · Evaluator

compare_results.py · Leaderboard · Heatmap · Radar · Dashboard





Cipher Factory

ciphers.py

```
class CaesarCipher(BaseCipher):  
    def encrypt(self, plain, shift=3): →  
        # shift each alpha char  
        chr((ord(c)-b+shift)%26+b)  
  
factory = CipherFactory()  
ct = factory.encrypt(text, "vigenere", key="SECRET")  
pt = factory.decrypt(ct, "vigenere", key="SECRET")
```

13 algorithms

registered in CipherFactory

Abstract base

BaseCipher enforces encrypt/decrypt

Roundtrip safe

All ciphers verified: `decrypt(encrypt(x))==x`

Extensible

New cipher = 1 class + `register()` call





LLMs Line Up

- Llama3
- Deepsek R1
- GLM 5
- GPT OSS 20B
- Kimi K2.5
- Qwen3.5-plus





Prompt Engineering: 4 Strategies

shot=0

Zero-Shot

No examples, no hints raw decryption task

"Decrypt this ciphertext and output ONLY the plaintext"

shot=1

Hint-Assisted

Algorithm name + description provided

"Encrypted with Caesar. Shift each letter backward by 3..."

shot=3

Few-Shot

3 plaintext ↔ ciphertext example pairs

Example 1: kloor → hello
Example 2: zruog → world

shot=9

Chain-of-Thought

Explicit step-by-step reasoning requested

1. Identify cipher
2. Decrypt step by step
→ PLAINTEXT: <r>text</r>





Evaluation Metrics

Exact Match

```
pred == ref
```

Strict identical string comparison

0 or 1

Near-Exact Match

```
lower(strip(pred)) == lower(strip(ref))
```

Catches UPPERCASE Morse/Polybius output

0 or 1

BLEU-1

```
 $\cap(\text{pred\_words}, \text{ref\_words}) / |\text{pred\_words}|$ 
```

Unigram word overlap (partial credit)

0.0 – 1.0

Char F1

```
 $2 \cdot P \cdot R / (P + R)$  at character level
```

Sensitive to small decryption errors

0.0 – 1.0

Cipher ID

```
keyword  $\in$  response
```

Did model identify cipher type?

0 or 1

Partial Credit

```
 $|\text{pred\_words} \cap \text{ref\_words}| / |\text{ref\_words}|$ 
```

Word-level partial decryption score

0.0 – 1.0





Python Toolkit

Data & I/O

- pandas
- jsonlines
- pydantic
- python-dotenv

Local Models

- transformers
- bitsandbytes
- accelerate
- torch

Visualization

- matplotlib
- seaborn
- numpy
- plotly

Evaluation

- rouge-score
- nltk
- scikit-learn
- python-Levenshtein

Concurrency

- asyncio
- ThreadPoolExecutor
- tqdm
- tenacity





Pipeline

1. Build Dataset

CipherFactory
encrypts
texts × 13 algos

2. Build Prompts

PromptEngine: 4
modes
with few-shot
examples

3. Call LLM

API / Ollama / HF
with retry + rate
limit

4. Score Response

extract_prediction()
near_exact_match
+ 5 metrics

5. Save JSON

results/{model}_shot{n}.json
CipherBank-compatible

6. Compare & Visualize

compare_results.py
Heatmap · Radar ·
Leaderboard

test.py

```
# Run one model across all shots – CipherBank style
for shot in [0, 1, 3, 9]:
    subprocess.run(['python', 'test.py', '--model', MODEL,
                    f'--shot_number={shot}'])
```





Handling Rate Limits



The Problem

API free tier: 15 req/min

21 of 39 requests failed (429)

Sequential execution: ~28 min

Model string typos → wrong errors



Python Fix

4s delay per request (rate_limit_delay)

30s inter-algo delay after each batch

Auto-retry ×2 on 429 + 60s backoff

ThreadPoolExecutor: 3× faster (async)

```
with ThreadPoolExecutor(max_workers=3) as ex:
    futures = {
        ex.submit(call_fn, task)
        for task in algo_tasks
    }
    for f in as_completed(futures):
        result = f.result()
```





Scoring Engine

prompt_engine.py

```
def extract_prediction(response, mode):
    # Priority 1: <r>text</r> tag
    m = re.findall(r"<r>(.*?)</r>", response)
    if m: return m[-1].strip()
    # Priority 2: labeled patterns
    # Priority 3: word overlap fallback

def near_exact_match(pred, ref):
    # Case-insensitive + strip punct
    p2 = pred.strip().lower()
    r2 = ref.strip().lower().rstrip(string.punctuation)
    return float(p2 == r2)

# Score: HELLO WORLD vs Hello World.
near_exact_match("HELLO WORLD",
                 "Hello World.") → 1.0 ✓
```

Why near_exact_match?

Morse and Polybius output uppercase text without punctuation.
Exact match fails → NEM captures the correct answer.

```
pred: "PHOTOSYNTHESIS CONVERTS
      LIGHT ENERGY INTO GLUCOSE"
ref : "Photosynthesis converts light
      energy into glucose."
→ EM=0.0 NEM=1.0 ✓
```

All metrics computed in Python:
extract → score → aggregate → export JSON



Results and Findings

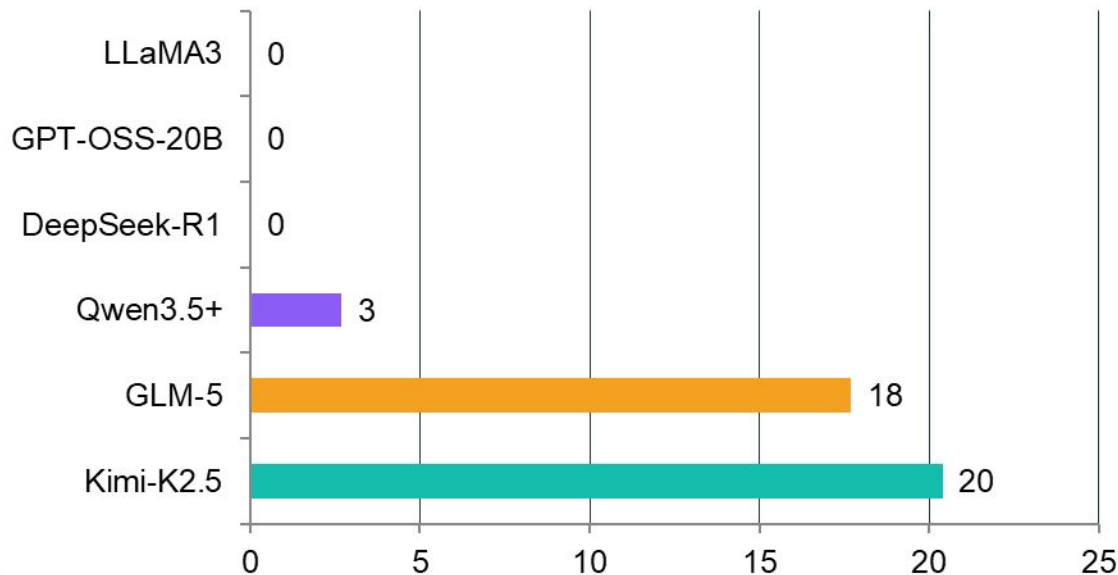
“



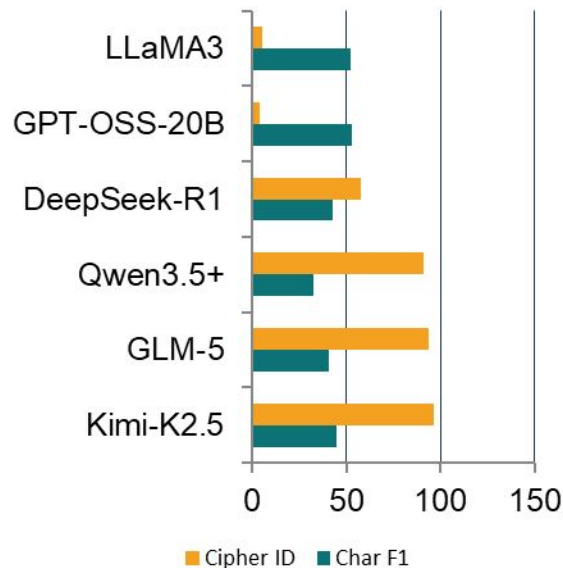


Overall Result

Avg Exact Match % (all shots, all algos)



F1 vs CID %



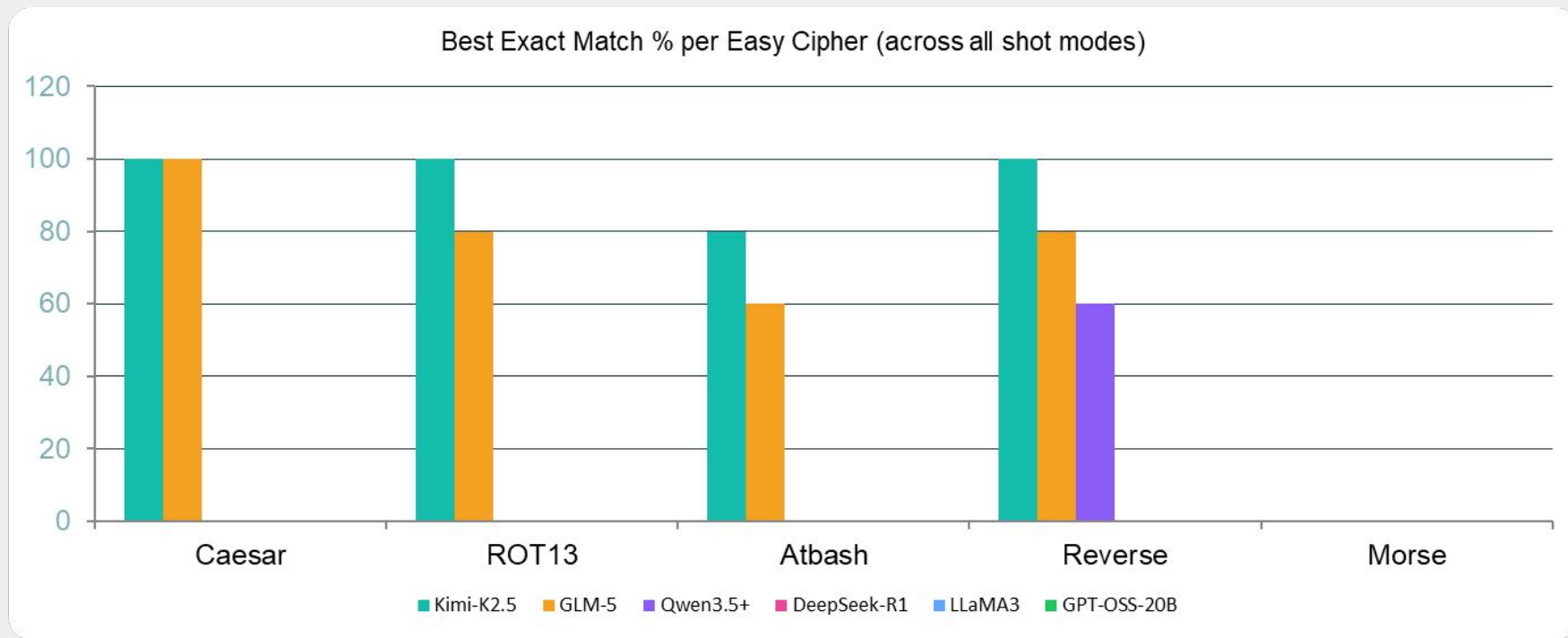
Kimi-K2.5 leads on EM (20.4%) · GPT-OSS-20B leads on F1 (53.3%) · Qwen3.5+ leads on Cipher ID (90.8%)



Pycon Busan
2026



Easy Tier Result



Kimi-K2.5 achieves perfect EM on Caesar, ROT13, Reverse. **GLM-5** perfect on Caesar. 3 models score zero across all Easy ciphers.



Easy Tier Result

Kimi-K2.5 — Caesar EM=1.0

Ciphertext : Qhxurqv wudqvplw hohfwulfdo vljqdov dfurvv vbqdswhf jdsb.

Reference : Neurons transmit electrical signals across synaptic gaps.

Response : "Let me try a Caesar shift. Looking at 'Qhxurqv' – if I shift back by 3:

Q→N, h→e, x→u, u→r, r→o, q→n, v→s → 'Neurons'..."

Prediction : Neurons transmit electrical signals across synaptic gaps.

GPT-OSS-20B — Reverse EM=0 F1=0.95

Ciphertext : .esoculg otni ygrene thgil strevnoc sisehtnysotohP

Reference : Photosynthesis converts light energy into glucose.

Response : "Potshneytoshtne ygrene thgil strevnoc otni .esoculg"

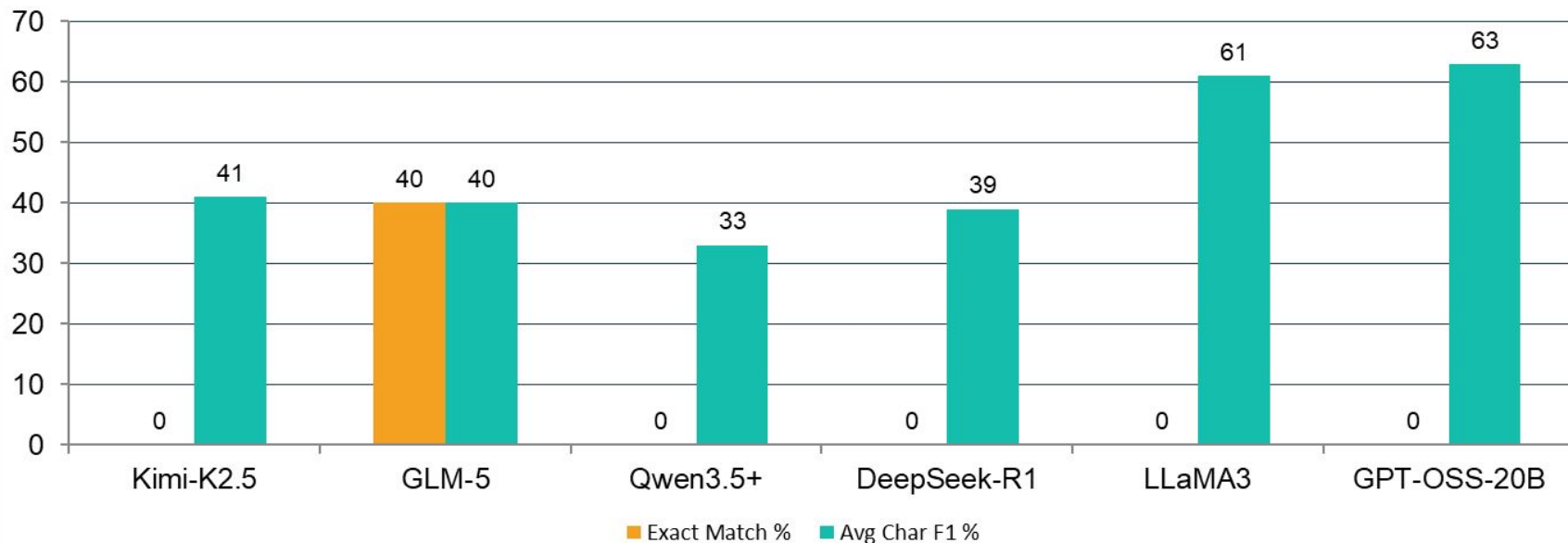
Prediction : Potshneytoshtne ygrene thgil strevnoc otni .esoculg





Medium Tier Result

Medium Tier — EM% vs Char F1% (best across shots)



⚠ Only GLM-5 achieves non-zero EM: Swap Pairs (40%) at shot=9

📊 F1 is high but EM=0 → Models produce plausible-but-wrong text



Medium Tier Result

GLM-5 — Swap Pairs EM=1.0 ✓

Ciphertext : hT ehtoeyro feralitivtc ahgndem dore nhpsyci soferev.r

Reference : The theory of relativity changed modern physics forever.

Response : "Algorithm: For each pair of letters, swap their positions...

| Pair | Swap Result |

hT → Th, eo → oe, th → ht, ey → ye, ro → or..."

Prediction : The theory of relativity changed modern physics forever. ✓

GPT-OSS-20B — Rail Fence EM=0 ✗ F1=1.00

Ciphertext : Potiorlte ouehtsnhsscneth nryit lcs.oye vsgegngo

Reference : Photosynthesis converts light energy into glucose.

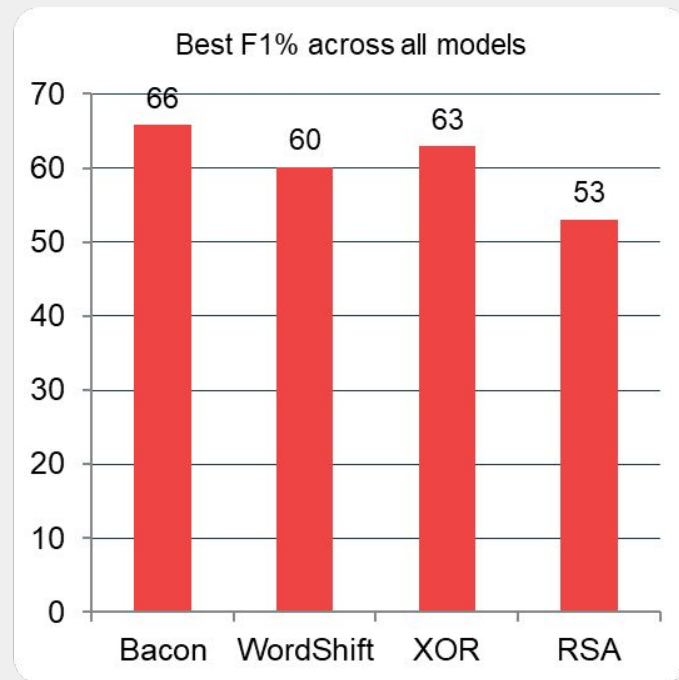
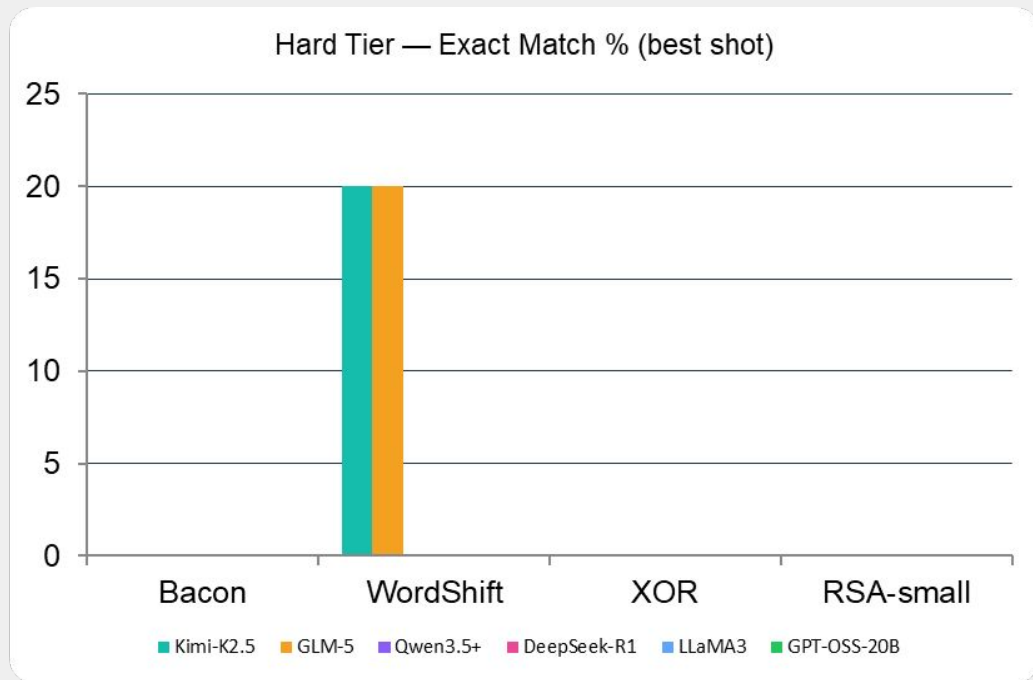
Response : "Potiorlte ouehtsnhsscneth nryit lcs.oye vsgegngo"

Prediction : Potiorlte ouehtsnhsscneth nryit lcs.oye vsgegngo





Hard Tier Result



Hard tier is the great equalizer. All 6 LLMs produce near-zero EM. Only WordShift sees any success (Kimi & GLM: 20%).





Hard Tier Result

Kimi-K2.5 — WordShift **EM=1.0**

Ciphertext : Skrwrvbqkwhlv gsrzivxw qnlmy ktkxme puav otckwam.

Reference : Photosynthesis converts light energy into glucose.

Response : "Context: 'Each word shifted by (3+word_index) positions.'

Word 0: 'Skrwrvbqkwhlv' → shift by 3 → S-3=P, k-3=h, r-3=o...

Word 1: 'gsrzivxw' → shift by 4 → g-4=c, s-4=o, r-4=n..."

Prediction : Photosynthesis converts light energy into glucose.

GPT-OSS-20B — XOR **EM=0** **F1=0.76**

Ciphertext :

133a36243b3c322b2d2b372a39276f282a3735372b24276f272c3e2b2679353a2a392220633b37243b6f

Reference : Photosynthesis converts light energy into glucose.

Response : "PLAINTEXT: Cryptography is the art of writing or solving codes."

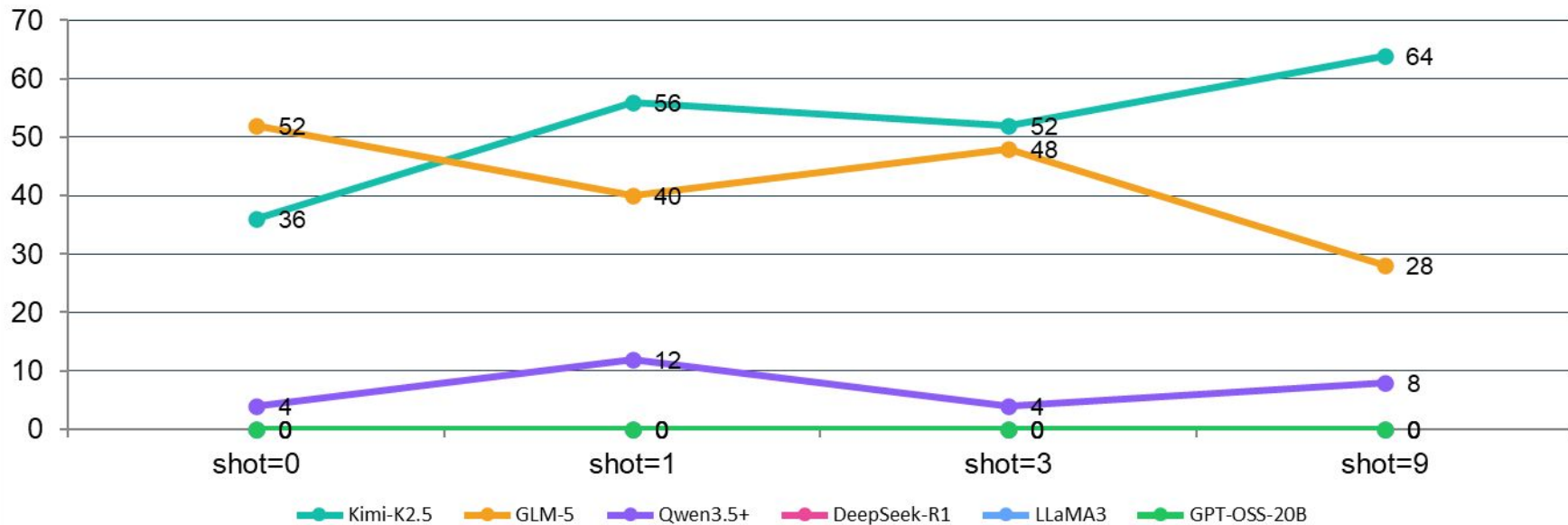
Prediction : Cryptography is the art of writing or solving codes.





Impact of Prompting Mode

Easy Tier EM % by Shot Mode

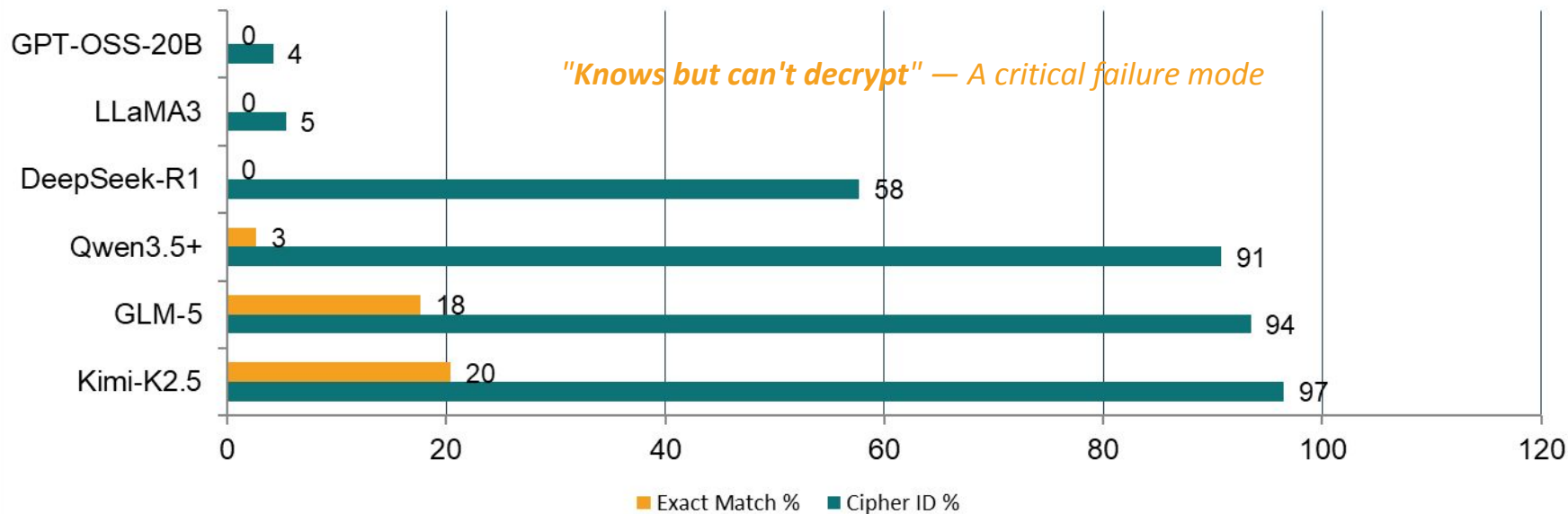


Kimi-K2.5 monotonically improves with more context (36%→64%). GLM-5 peaks at zero-shot (52%) and declines — suggesting hint-mode confuses it. Others flat at 0%.



CID vs Exact Match

Cipher ID% vs Exact Match% — averaged across all shots & algos



 Kimi: CID=96.5% but EM=20.4% → identifies cipher but often fails to execute

 LLaMA3: CID≈5% doesn't even mention cipher type in 95% of responses



Pycon Busan
2026



Key Findings

01

Tier Stratification Works

Easy ciphers effectively separate high-capability from low-capability models. Kimi & GLM succeed; 4 models score zero — confirming difficulty tiering as a valid evaluation design.

02

Hard Tier Remains Unsolved

No model achieves >20% EM on any Hard cipher. WordShift (20%) is the outlier. XOR and RSA remain at 0% EM universally — mathematical reasoning in zero-shot context is insufficient.

03

Know vs. Do Gap is Real

Kimi (CID=96.5%) identifies ciphers in nearly every response yet decrypts correctly only 20% of the time. GLM and Qwen show similar patterns — awareness ≠ execution.

04

Prompting Has Uneven Effects

More context helps Kimi (+28pp Easy). But GLM declines with hints (-24pp). DeepSeek and LLaMA show virtually no response to prompting changes — model architecture matters more than prompt.

05

F1 ≠ Progress for Some Models

LLaMA3 and GPT-OSS show high F1 (52–63%) with zero EM — indicating structural/format failures (no <r>tag, wrong case) rather than actual decryption errors. Near-Exact Match would likely reveal more true successes.

06

Provider Architecture Matters

API-served models (Kimi, GLM, Qwen) outperform models (LLaMA3, GPT-OSS, DeepSeek) on EM. Local serving or quantization may degrade response quality.





Anomali and Surprising



DeepSeek-R1: Hard > Medium F1

A reasoning model scores higher Char F1 on Hard tier (40.5%) than Medium (38.4%). RSA-small F1=53% is surprisingly high possibly because integer token sequences overlap with any numerically-rich reference text.



GLM-5: CoT Harms Performance

GLM-5 Easy EM falls from 52% (zero-shot) to 28% (CoT). Additional instructions appear to confuse the model or introduce output format changes that fail the extractor contradicting the conventional wisdom that CoT improves accuracy.



LLaMA3 Bacon F1=65.8%

LLaMA3 achieves the highest Bacon cipher F1 across all models (65.8% at CoT) despite zero EM. The model may be outputting the correct character structure in a non-standard format that the extractor fails to parse.



Qwen3.5+ CID Without Decryption

Qwen achieves 90.8% average CID but only 2.7% EM the largest identification-to-execution gap. Qwen may be outputting detailed explanations of how ciphers work without actually applying the decryption algorithm to the input.





Failure Mode Taxonomy



Format Mismatch

LLaMA3, GPT-OSS, Morse/Polybius

Model decrypts correctly but outputs uppercase, adds explanation text, or omits `<r>` tags. High F1, zero EM. Fix: enforce output format strictly.

~40%



Hallucination

All models on Medium/Hard

Model produces plausible-sounding but wrong plaintext. Char F1 is non-trivially positive (0.3–0.5) but content is incorrect. Pattern-matches to common English text.

~35%



Partial Decryption

All models, long texts

First few words correct, then model drifts. Char F1 declines with text length. Model loses track of cipher state mid-decryption.

~15%



Knowledge-Execution Gap

Kimi, GLM, Qwen primarily

Model identifies cipher type (CID=1) but fails to apply the algorithm correctly. Discusses decryption methodology without executing it. Most visible in Hard tier.

~8%



Mathematical Failure

All models on XOR, RSA

Cipher requires integer arithmetic (XOR bitwise ops, modular exponentiation). Model cannot compute $\text{pow}(m, e, n)$ in-context — needs tool-use or code execution.

~2%



Conclusion

01 Kimi-K2.5 leads — but only on Easy

Best EM overall (20.4%). Perfect scores on Caesar, ROT13, Reverse. No model solves Medium/Hard reliably at scale.

02 The Know-Do Gap Is Universal

All models with high CID (Kimi: 96.5%, GLM: 93.5%) still fail to decode. Understanding cipher logic $\neq$ executing it.

03 Prompting Has Diminishing Returns

Shot mode matters for top models (Kimi: +28pp) but has near-zero effect on LLaMA3, DeepSeek, GPT-OSS.

04 Hard Tier Exposes Fundamental Limits

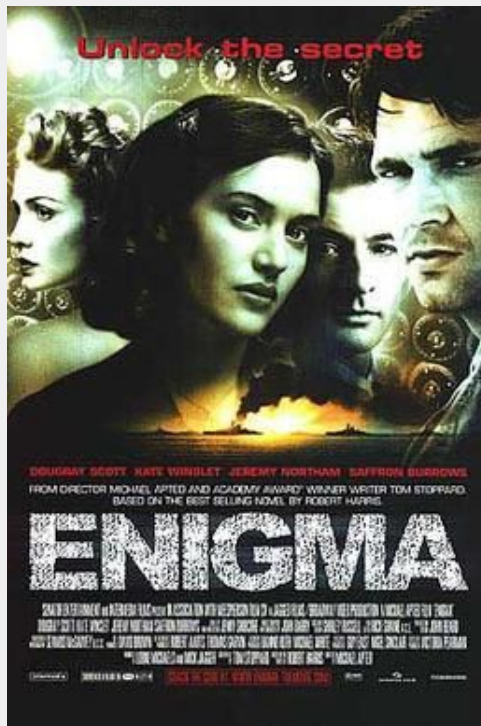
Mathematical ciphers (XOR, RSA) are universal failures. Current LLMs cannot perform in-context bitwise or modular arithmetic reliably.

Humans Still Lead on Hard Tasks : Mathematical cryptanalysis remains an open challenge for LLMs.





Highly recommended to watch



Thank You!

